

SINIFLAR ARASI İLİŞKİLER VE SINIF TASARLAMA İLKELERİ

Unified Modeling Language

Unified Modeling Language (UML) nesneye dayalı yazılım geliştirme ile gelişen bir modelleme aracıdır. Bu nedenle Nesne Yönelimli Programlama başlığında açıklanan nesne yönelimli birçok kavram, izleyen sayfalarda daha da pekişecektir. Adından da anlaşılacağı üzere nesneye dayalı problem çözmenin kalbi, model oluşturmaktır.

Model, gerçek dünyadaki karmaşık problemin esaslarını soyut olarak önümüze koyar. Buradan hareketle UML'i, basit bir dil, gösterim ya da sözdizimi (syntax) gibi algılamalıyız. UML'in, yazılımı nasıl geliştireceğimizi kesin olarak belirtmediği göz önüne alınmalıdır.

UML, bir grafik modelleme dili olup yazılım sistemlerini oluşturan ana elemanları (Ki bunlar İngilizce "artifact" olarak adlandırılır ve el yapımı anlamına gelmektedir.) çeşitli şekillerden oluşacak şekilde tanımlamak için oluşturulmuş bir kurallar bütünüdür. Örnek verecek olursak, bir mimari projeye ilişkin maket ne ise yazılım için UML de aynı şeyi ifade eder. Yani UML, söz konusu yazılımın neyi yapacağını anlatan çeşitli şekillerin anlamlı bir şekilde bir araya getirilmesini sağlayan bir standarttır.

Yazılım mühendisliğiyle (software engineering) birlikte 1989 ve 1994 yılları arasında elliden fazla modelleme dili kullanılmaya başlanmıştır. Bu nedenle bu dönem yöntem savaşlarının yaşandığı bir dönem olarak adlandırılır. Bu yöntemlerin çoğu kendi sözdizimini oluşturmuş olmakla birlikte kullandıkları dil elemanları açısından birbirleriyle benzerlik göstermektedirler.

Doksanlı yılların ortalarında üç yöntem, daha güçlü ve yaygın hale gelmiştir. Bu yöntemlerin her biri, diğerlerinin içerdiği elemanları da içeriyordu ancak her birinin diğerine karşı çeşitli güçlü özellikleri bulunuyordu. Bu yöntemler:

- *Booch* yönteminin **tasarım (design)** ve **kodlama (implementation)** yönleri oldukça iyi idi. *Grady Booch* Ada diliyle oldukça çalışmıştı ve nesne yönelimli tekniklerin Ada diline uygulanması konusunda yaptığı çalışmalarla bu konuda önemli bir oyuncu olmuştur. Booch metodu güçlü olmasına rağmen model, birçok bulut şeklinden oluşmasından dolayı gösterim şekli oldukça sevimsizdi.
- Object Modeling Technique (OMT) yöntemi, *Jim Rumbaugh* tarafından geliştirilmiş olup, analiz ve veri merkezli bilgi sistemlerinin modellenmesinde oldukça iyi bir yöntemdir.
- Object Oriented Software Engineering (OOSE) modeli use-case olarak bilinen bir model olup *Ivar Jacobson* tarafından geliştirilmiştir. Use-case modeli, yazılımı yapılacak sistemin davranışlarını anlamaya yönelik gelişmiş güçlü bir tekniktir.

1994 yılında *Jim Rumbaugh* ve General Electric firmasından ayrılan *Grady Booch* birlikte Rational şirketini kurarak bir araya geldiler. Bu birlikteliğin amacı, kendi yöntemlerini bir araya getirecek yeni ve tek bir yöntem olan "Unified Method"u oluşturmak idi.

1995 yılında *Ivar Jacobson* da Rational şirketine katıldı. *Ivar Jacobson*'ın use-case'ler konusundaki fikirleriyle birlikte Rational şirketi, bugün Unified Modeling Language-UML olarak adlandırılan, yeni bir "Unified Method" geliştirdi. Üç kişiden oluşan bu grup "Three Amigos" olarak bilinir.

Yöntem savaşlarının ardından güçlü yazılım üreticileri bir araya gelerek OMG adında bir şirketler birliği kurdular¹⁹. Bu şirketler birliği 1997 yılında UML'e sahip çıkarak herkes tarafından kullanılan bir dil olmasını ve bir standart haline gelmesini sağlamıştır.

UML sadece, gerçek dünyadaki süreçlerin modelini ortaya koyan bir **gösterim (notation)** sistemidir. Süreçlerin standartlaştırılmasına yönelik bir dil değildir. UML ile ortaya çıkan modeller, süreçleri **somut**

¹⁹ OMG (Object Management Group), www.omg.org

(**abstract**) olarak tanımlayacaktır, ancak süreçlerin doğruluğu hakkında bir bilgi vermeyecektir. Yani, A şirketi için çalışan bir süreç, B şirketine uygulandığında felaketle sonuçlanabilir.

Model, yukarıda da anlatıldığı üzere çözülecek probleme ilişkin bir **soyutlamadır** (**abstraction**). Etki alanı (domain) ise problemin yaşandığı gerçek dünyayı ifade eder.

Model, birbirlerine **ileti** (**message**) gönderen nesnelerden (object) oluşur. Bu modelde nesneler **canlı** (**alive**) olarak düşünülmelidir. Nesneler bir şeyler bilir (**know**) ve bildikleriyle bir şeyler yaparlar (**do**).

Modelde nesneler, nesne tanımında verilen **özellik** (**property**) ve **alanları** (**field**) gösteren **niteliklere** (**attribute**) sahiptirler. Nesnelerin gösterdikleri **davranış** (**behavior**) ya da geliştirdiği **yöntemler** (**method**) ise işlem (operation) olarak adlandırılır. Nesnenin özelliklerine ait o anki değerler ise nesnenin **durumunu** (**state**) belirler.

UML, sekiz tür diyagrama sahiptir ve bu kitapta sınıf diyagramları anlatılacaktır²⁰.

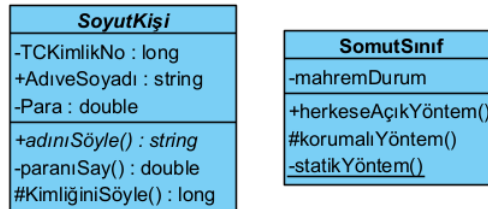
Sınıf Diyagramları

UML diyagramlarından biri olan **sınıf diyagramları** (**class diagram**), yazılımı geliştirilecek olan sisteme ait **sınıfları** (**class**) ve bu sınıflar arasındaki **ilişkiyi** (**relationship**) gösterir. Sınıf diyagramları durağandır (**static**). Yani sadece sınıfların birbiriyle etkileşimini (**what interacts**) gösterir, bu etkileşimler sonucunda ne olduğunu (**what happens**) göstermez.

Sınıf diyagramlarında sınıflar, bir dikdörtgen ile temsil edilir. Bu dikdörtgen üç parçaya ayrılır. En üstteki birincisine sınıf ismi yazılır. Ortadaki ikincisinde **nitelikler** (**attribute**), en alttaki ve sonuncusunda ise ve **işlemler** (**operation**) yer alır.

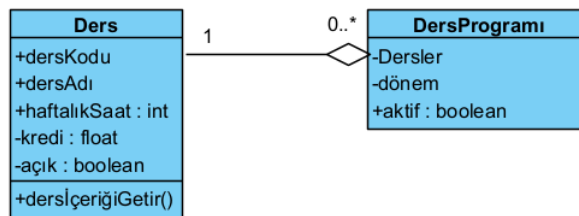
Sınıf isminin italik yazılması, sınıfın **soyut** (**abstract class**) sınıf olduğunu gösterir. Benzer şekilde işlemlerin yer aldığı kısımda, yöntemlerin italik yazılması halinde, ilgili yöntemin **soyut yöntem** (**abstract method**) olduğunu gösterir.

Sınıflarda **erişim belirleyiciler** (**access modifier**), özel karakterlerle birbirinden ayrılır. Burada “-” karakteri **mahrem** (**private**), “+” karakteri **umuma açık** (**public**), “#” karakteri ise **korunmalı** (**protected**) olduğunu anlatmaktadır. **Statik üyeler** (**static member**), altı çizili olarak gösterilir.



Şekil 26. Örnek Bir Sınıf Diyagramı

Çokluk (**multiplicity**), ilişkide bir sınıfın **örnek** (**instance**) sayısını gösterir. İlişkiyi gösteren çizginin sonunda bir numara olarak gösterilir.



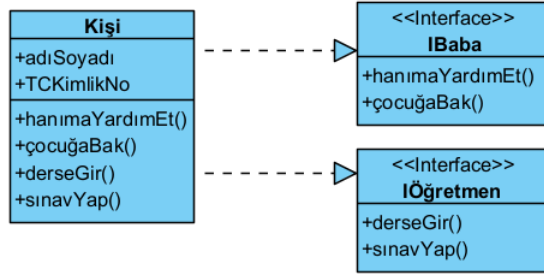
Şekil 27. Sınıf Diyagramlarında Sınırlama ve Çokluk Örneği

Bu numara, mümkün olabilecek örnek sayısıdır. Çokluk bir sonraki başlıkta verilen, **iş birliği** (**association**), **bileşim** (**composition**) ve **bütünleşmeye** (**aggregation**) uygulanır. Çokluk örnekleri:

- **0..1** Sıfır ya da 1 örnek.
- **0..*** Veya ***** Örnek sayısı sınırsızdır.

²⁰ www.togethersoft.com/services/practical_guides/umlonlinecourse/index.html

- 1 Yalnızca 1 örnek
- 1.. En az bir örnek



Şekil 28. Sınıf Diyagramlarında Ara yüz Gerçekleştirme

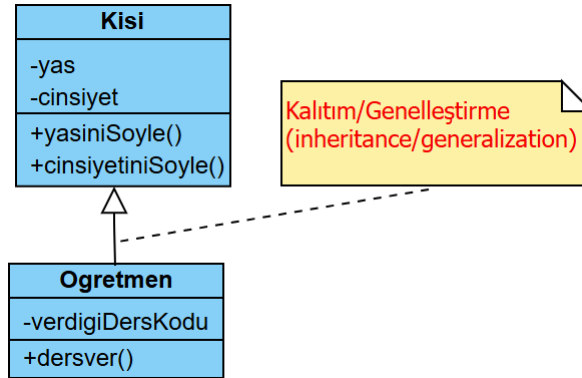
Ara yüz gerçekleştirme (interface realization) işlemi, ucunda üçgen olan kesikli çizgi ile gösterilir. Üçgenin sivri köşesi gerçekleştirilen ara yüzü gösterir. Çemberler, ara yüzleri (interface) göstermenin bir başka yoludur. Bu durumda daha kısa ve etkili anlatıma sahip diyagramlara sahip oluruz. Bu tip basitleştirilmiş diyagramlar *lollipop diyagramı* olarak da adlandırılır.

Sınıflar Arası İlişkiler

Nesneleri imal ettiğimiz sınıflar arasında ya ilişki yoktur ya da aşağıda sıralanan ilişkiler vardır;

Genelleştirme İlişkisi

Genelleştirme (generalization), taban sınıfla (base class) ile türemiş sınıf (derived class) arasındaki kalıtım (inheritance) ilişkisidir. Örneği *Kalıtım* başlığında verilmiştir. Aşağıda Kişi sınıfı ile bu sınıftan miras alan Öğretmen sınıfının UML diyagramı verilmiştir. Ara yüzler (interface) veya roller de benzer şekilde gösterilir.

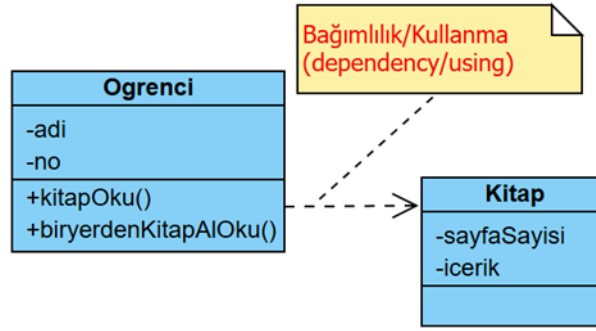


Şekil 29. Genelleştirme İlişkisi UML Diyagramı

Bağımlılık İlişkisi

Bağımlılık (dependency) ya da bir başka deyişle kullanma (using), bir sınıf ile diğer arasındaki geçici ilişkidir. Burada kullanan sınıf *istemci* (client) diğer sınıf ise *sağlayıcı* (supplier) olarak adlandırılır. İki türlü bağımlılık vardır;

- İstemci sınıf, sağlayıcıyı sınıfı *parametre olarak kullanır* (dependency as a parameter).
- İstemci sınıf, sağlayıcı sınıftan *örnekleme yapar* (dependency as an instantiation).



Şekil 30. Bağımlılık ilişkisi UML Diyagramı

```

// See https://aka.ms/new-console-template for more information
Öğrenci ali = new Öğrenci();
Kitap birkitap = new Kitap();
ali.kitapOku();
ali.biryerdenKitapAlOku(birkitap);

public class Kitap
{
    public int SayfaSayısı { get; set; }
    public string İçerik { get; set; }
}
public class Öğrenci
{
    public string Adı { get; set; }
    public int No { get; set; }
    public void kitapOku()
    {
        Kitap kitap = new Kitap(); // dependency as an instantiation
        Console.WriteLine($"{kitap.İçerik}");
    }
    public void biryerdenKitapAlOku(Kitap kitap) // dependency as a parameter
    {
        if (kitap.SayfaSayısı > 100)
        {
            /* Yaratılacak Öğrenci Nesneleri bir başka yerde
             * imal edilmiş "kitap" nesnesini parametre olarak
             * alarak kullanıyor. */
            Console.WriteLine("Uzun Kitap...");
        }
    }
}

```

Örnekteki Öğrenci ve Kitap sınıfı arasındaki bağımlılık ilişkisi aşağıdaki UML diyagramında verilmiştir.

İş Birliği İlişkisi

Ortaklık olarak da adlandırılan **iş birliği** (*association*), iki farklı sınıf arasındaki sürekli olan **bütün-parça** (*whole-part*) ilişkisidir. Bu ilişkide, bir sınıf diğer sınıftan nesne örnekler (imal eder), kullanır ve öldürür. İlişkinin sürekli olabilmesi için kullanılan sınıftan bir değişken **alan** (*field*) olarak tanımlanır. İki türü vardır;

- **Açık iş birliği** (*public association*): Kullanılan sınıfa ilişkin değişken **public** erişimi ile tanımlanır.
- **Gizli iş birliği** (*private association*), **içerme** (*containment*) ya da **bileşim** (*composition*): Kullanılan sınıfa ilişkin değişken **private** erişimi ile tanımlanır ve dışarıdan erişime izin verilmez.

```

// See https://aka.ms/new-console-template for more information
Öğretmen ilhan = new Öğretmen();
ilhan.kitapOku(); // ilhanın kitabına erişilemez!
Öğrenci ali = new Öğrenci();
ali.kitapOku(); // alinin kitabına dışarıdan erişilebilir!
Kitap alininkitabı = ali.kitap;

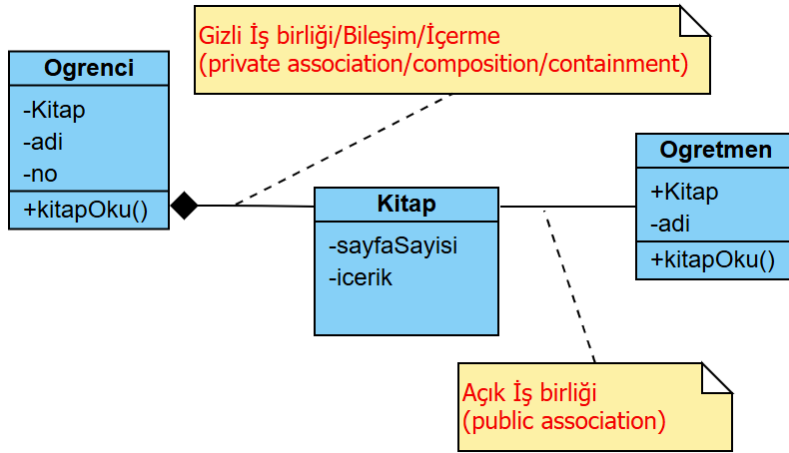
```

```

public class Kitap
{
    public int SayfaSayısı { get; set; }
    public string İçerik { get; set; }
}
public class Öğrenci
{
    public string Adı { get; set; }
    public int No { get; set; }
    public Kitap kitap;
    public Öğrenci()
    {
        kitap = new Kitap();
    }
    public void kitapOku()
    {
        Console.WriteLine($"Öğrenci Kitabını Okuyor...{kitap.İçerik}");
    }
}
public class Öğretmen
{
    public string Adı { get; set; }
    public int No { get; set; }
    private Kitap kitap;
    public Öğretmen()
    {
        kitap = new Kitap();
    }
    public void kitapOku()
    {
        Console.WriteLine($"Öğretmen Kitabını Okuyor...{kitap.İçerik}");
    }
}

```

Yukarıdaki örnekte Öğrenci ile Kitap arasındaki gizli ortaklık ile Öğretmen ve Kitap arasındaki açık iş birliği aşağıdaki UML diyagramında gösterilmiştir;

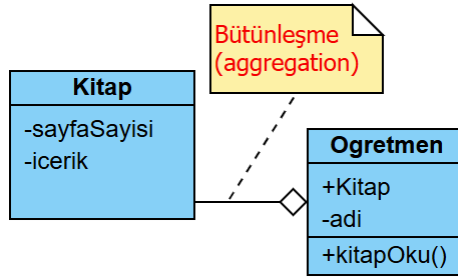


Şekil 31. Açık ve Gizli İş Birliği UML Diyagramı

UML diyagramlarında, çift yönlü ortaklıklar iki oka sahip olabilir veya hiç ok olmayabilir ve tek yönlü ortaklıkta veya kendi kendine ortaklık bir oka sahiptir.

Bütünleşme İlişkisi

Bütünleşme (**aggregation**), ilişkisi de iki farklı sınıf arasındaki sürekli olan **bütün-parça** (**whole-part**) ilişkisidir. **Açık ortaklığa** (**public association**) benzer, farklı olarak bu ilişkide kullanılan sınıfa ait başka yerde imal edilmiş nesneyi ya da hazır olan nesneyi kullanır. Tek fark kullanılacak sınıf nesnesinin kullananın iradesi dışında yaratılmış olmasıdır.



Şekil 32. Bütünleşme UML Diyagramı

```

// See https://aka.ms/new-console-template for more information
Kitap birkitap = new Kitap();
Öğretmen ilhan = new Öğretmen(birkitap); // Öğretmen Kitapsız imal edilemez!
ilhan.kitapOku();

public class Kitap
{
    public int SayfaSayısı { get; set; }
    public string İçerik { get; set; }
}

public class Öğretmen
{
    public string Adı { get; set; }
    public int No { get; set; }
    private Kitap kitap;
    public Öğretmen(Kitap kitap)
    {
        this.kitap = kitap;
    }
    public void kitapOku()
    {
        Console.WriteLine($"Öğretmen Kitabını Okuyor...{kitap.İçerik}");
    }
}
  
```

Yukarıdaki örnekte verilen Öğretmen ve Kitap arasındaki bütünleşme ilişkisi aşağıdaki UML diyagramında gösterilmiştir.

Sınıf Tasarlama İlkeleri

Nesne yönelimli programlamada (**object oriented programming**); Kodumuzda değişim yönetimi (**change management**), gösterici kullanımı yerine referansları kullanarak güvenli kod (**safe code**) aynı kodları tekrar yazmak (**duplicate code**) yerine kodların yeniden kullanımı (**reusing**) sağlayan aşağıdaki özellikleri kullanırız;

- Kod küçüldüğü için **kalıtım** (**inheritance**),
- Veri ve kontrol **soyutlaması** (**abstraction**) yaparak daha güvenli bileşen tasarladığımız için **sarmalama** (**encapsulation**),
- Yazılım mimarisinin daha esnek ve genişleyebilir olması için **çok biçimlilik** (**polymorphism**).

Ayrıca yazılım geliştirmede;

- **Zayıf bağlaşım** (**loosely coupling**) sağlanması için yazılımın bir noktasında olabilecek değişikliğin, diğer kısımlarda değişiklik gerektirmeyecek şekilde tasarlanması gerekir. Yani yapılan değişiklik, çorap söküğü gibi yazılımın diğer kısımlarını değiştirmemelidir.
- Aynı çağrışım kümesine ilişkin bütün bileşenler bir arada geliştirilmeli diğer kümelerle karıştırılmamalıdır (**seperation of concern**). Yani bileşenler arasında **yüksek uyum** (**high cohesion**) olmalıdır.

İşte yeniden kullanım, zayıf bağlaşım ve yüksek uyum sağlamak için programcı, nesne yönelimli programlama özellikleriyle birlikte temel **ilkeler** (**principle**) uyar.

Programcı kodu tasarlarken, yazarken ve yeniden düzenlerken bu ilkeleri aklında tutarsa; Kod çok daha temiz, genişletilebilir ve test edilebilir olur.

Bunlar ilkelerin İngilizce baş harflerinin alınarak isimlendirilen SOLID ilkeleridir (**SOLID Principles**);

Tek Sorumluluk İlkesi

Bu ilke (Single Responsibility Principle-SRP): sınıflar, tasarlanırken birden fazla işten sorumlu tutulmamalıdır. Mümkün olduğunca bir işle ilgili olarak tasarlanmalıdır. Bir sınıfın tek bir şey yapması gerektiğini ve dolayısıyla değişmesi için tek bir nedeninin olması gerektiğini belirtir.

Açık-Kapalı İlkesi:

Bu ilke (Open Closed Principle-OCP) sınıflar, değişikliklere kapalı (**closed for modification**) eklentilere açık (**open for extension**) şekilde tasarlanmalıdır. Bu şekilde değişikliklere açık genişleyebilir modüllere sahip oluruz. Değişikliklere kapalı olmak, mevcut bir sınıfın kodunu değiştirmemek anlamına gelirken, eklentilere açık olma yeni işlevler eklemek anlamına gelir.

Liskov İkame İlkesi

Bu ilke (Liskov Substitution Principle-LSP) alt sınıflar türediği sınıfların yerine konulabilmelidir. Bu prensipten ilk olarak Barbar Liskov tarafından bahsedilmiştir. Alt sınıflar, türediği sınıf davranışlarıyla kullanılabilir. Burada amaç, alt sınıfların daha çok değişebileceği göz önüne alınarak değişimlerden en az şekilde etkilenmektir.

Türemiş sınıftan bir nesneyi, Taban sınıftan bir nesne bekleyen herhangi bir yöntem parametre olarak geçirdiğimizde yöntemin herhangi bir tuhaf çıktı vermemesi gerektiği anlamına gelir. Bu beklenen davranıştır, çünkü kalıtım kullandığımızda alt sınıfın, üst sınıfın sahip olduğu her şeyi miras aldığını varsayabiliriz. Alt sınıf davranışı genişletir ancak asla daraltmaz. Dolayısıyla bir sınıf bu ilkeye uymadığında, tespit edilmesi zor bazı kötü hatalara yol açar.

Ara Yüzleri Ayırma İlkesi

Bu ilke (Interface Segregation Principle-ISP) genel amaçlı ara yüzler yerine istemciye bağlı ara yüzler tasarlanmalıdır. Yani roller, birbirinden ayrı olacak şekilde tanımlanmalıdır. Birden fazla rol tek bir rol altında olacak şekilde düşünülmemelidir. Bu nedenle buna rollerin ayrılması prensibi (**Role Segregation Principle-RSP**) olarak da adlandırılır.

Bağımlılıkları Ters Çevirme İlkesi

Bu ilke (Dependency Inversion Principle-DIP) sınıflarımızın somut sınıflara ve fonksiyonlara değil, ara yüzlere veya soyut sınıflara bağlı olması gerektiğini belirtir. Nesne yönelimli programlamanın en önemli ilkesidir. Aslında bu ilke ile açık-kapalı ilkesi doğrudan birbiriyle ilişkilidir.